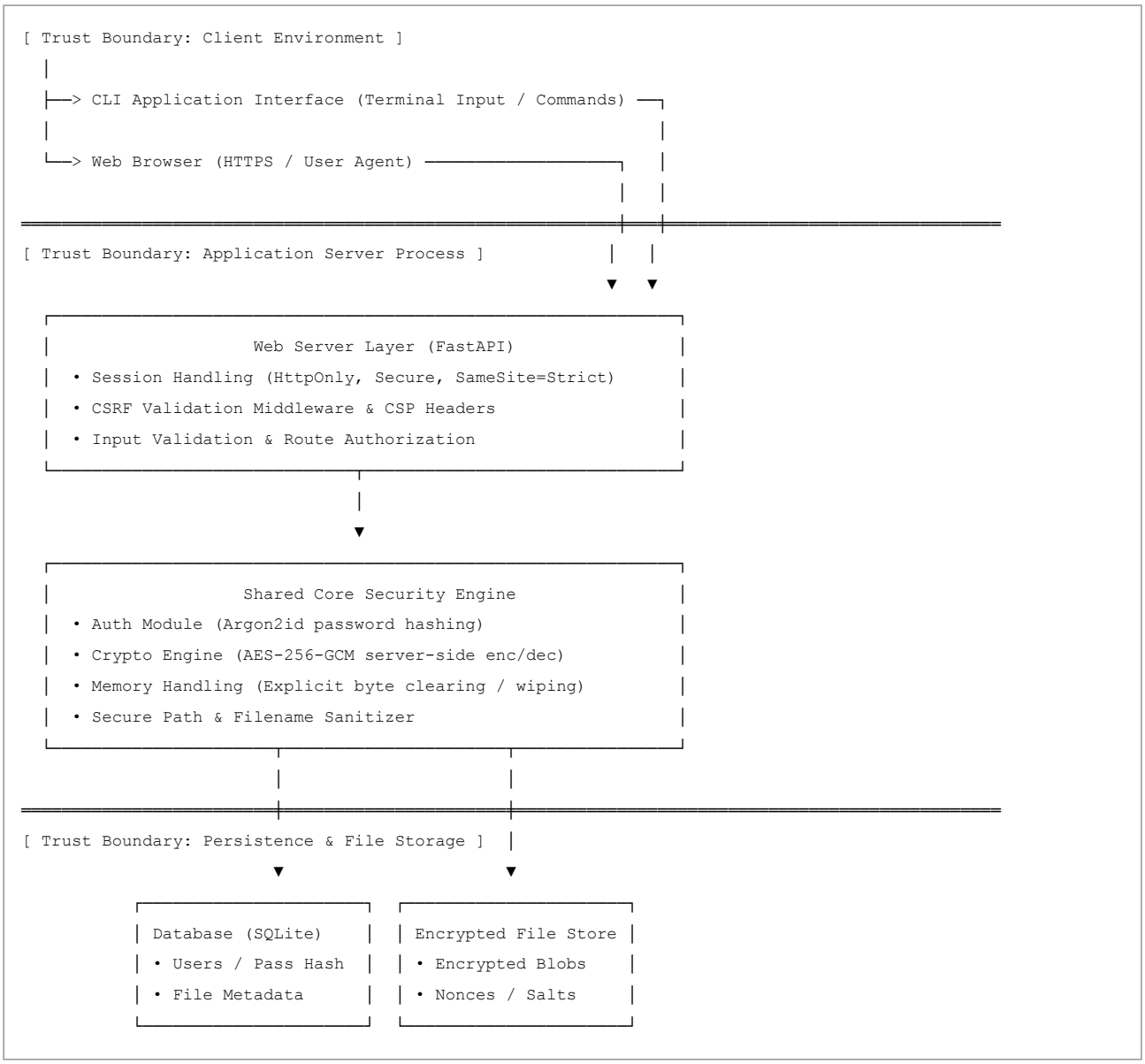
Checkpoint 1: Design Document

1. System Architecture and Component Diagram

The application has a low-level core package (`core/`). The package contains the cryptographic engine, storage manager, and memory handling logic.

Two entry points use this package: a **CLI executable** for ICS0022 and an **HTTP API web application** for ICS0027.



Data Flow and Encryption Locations

1. **In-Transit Protection:** TLS 1.3 encrypts all web traffic. The CLI handles traffic in process memory.
2. **Key Derivation and Storage:** Argon2id derives keys from master passwords. The system derives an encryption key for each file and user operation. The system stores each key in a temporary memory buffer. The system clears each buffer after the cryptographic operation.
3. **Data-at-Rest Encryption:** The server or core engine encrypts each file with **AES-256-GCM** before it writes the file to the target path.

---

2. Combined Threat Model & Mitigations

This threat model covers OWASP Top 10 web vulnerabilities, memory security, and CLI security:

Domain	Threat / Vulnerability	Course Target	Intended Mitigation
Memory	Buffer Overflows and Memory Leaks	ICS0022	Use strict byte and length limits. Use safe memory allocation. Clear buffers that contain keys or passwords after each operation.
XSS	HTML/JavaScript Injection	ICS0027	Encode all output. Use Content Security Policy (`default-src 'self'`). Escape output for its context in the web interface.
CSRF	Cross-Site Request Forgery	ICS0027	Add a separate anti-CSRF token to each session. Include the token in each web request that changes state. Set the `SameSite=Strict` and `HttpOnly` cookie flags.
Injection	SQL Injection	Both	Use parameterized queries or the SQLAlchemy ORM for all database access.
Input/Path	Path Traversal and Command Injection	Both	Sanitize all file names. Map stored files to UUIDs or hashes. Reject relative paths such as `../`. Do not call a shell.
Auth	Session Fixation and Hijacking	ICS0027	Create a new session ID at login. End the session at logout or timeout. Set the `Secure`, `HttpOnly`, and `SameSite` cookie flags.
Storage	Unencrypted Secret or File Leakage	Both	Use AES-256-GCM authenticated encryption for files at rest. Store only Argon2id password hashes and cryptographic salts in the database.
Bypass	Client-Side Control Bypass	ICS0027	Enforce authorization and input validation on the server. Repeat or replace client checks at backend endpoints.

---

3. Technology Stack & Justification

- **Language: Python 3.11+**
  - *Reason:* Python has a readable syntax and a standard memory abstraction. It also supports explicit byte clearing with `bytearray` or custom memory buffers. Python provides web and CLI libraries.
- **Cryptographic Library: `cryptography` (PyCA)**
  - *Reason:* This Python library provides primitives such as `AESGCM` and `Argon2id`. It uses C-level OpenSSL bindings.
- **Web Framework: FastAPI (Uvicorn)**
  - *Reason:* This asynchronous web framework provides automatic OpenAPI documentation, Pydantic validation, and middleware for session management.
- **Database: SQLite (via SQLAlchemy ORM)**
  - *Reason:* This lightweight file database suits local semester deployment. It supports parameterized queries.
- **TLS Deployment Plan:**
  - The local development server will use HTTPS. `mkcert` or `OpenSSL` will create a self-signed certificate. Uvicorn will load the certificate with `--ssl-keyfile` and `--ssl-certfile`.

---

4. Planned Authentication & Session Model

- **Password Hashing:** Use Argon2id with a unique salt for each user.
- **Web Session Security:**
  - **Cookies:** Store the session ID in a cookie with the `HttpOnly` flag. This flag blocks JavaScript access. Set the `Secure` flag to require HTTPS. Set the `SameSite=Strict` flag to reduce CSRF risk.
  - **Session Lifetime:** Set the idle timeout to 15 minutes. Set the absolute expiry to 12 hours.
  - **Fixation Prevention:** Destroy the existing session ID after successful credential verification at login. Create a new session ID at that time.
- **CLI Authentication:**
  - Prompt for credentials when the user starts the program. Keep session state only while the process runs. Derive the master key in memory. Clear the key when the process exits.

Secure Web and CLI File Encryption System

This file management system has two interfaces. It meets the requirements for **ICS0027 (Web Application Security)** and **ICS0022 (Secure Programming)**.

Scope and Features

- **Core Security:** Encrypt files at rest with AES-256-GCM. Hash passwords with Argon2id. Clear memory after each cryptographic operation.
- **Web Interface (ICS0027):** Use CSRF protection, CSP headers, XSS prevention, and HTTPS session management.
- **CLI Interface (ICS0022):** Manage files in a terminal application with secure input handling and memory safety.

CLI Commands (ICS0022)

Run as `python -m src.cli.main <command>` (the program name is `secfile`). Every command that needs an account prompts for the password with echo disabled (`-u/--username` selects the account).

- `secfile register -u <user>` - Create a user account
- `secfile login -u <user>` - Verify credentials
- `secfile encrypt -u <user> -i <file> -o <name>` - Encrypt a local file (`-i`) into the user's encrypted store under the name `-o`
- `secfile decrypt -u <user> -i <name> -o <out> [-f]` - Decrypt a stored file (`-i`, name as shown by `list`) to the path `-o` (`-f` overwrites an existing file)
- `secfile list -u <user>` - List encrypted user files
- `secfile wipe <file> [-y]` - Delete a local file securely

Files stored via the CLI and via the web interface share one per-user store, so a file encrypted in one interface can be decrypted in the other.

Input validation: input paths must be readable regular files up to 50 MiB; output paths must have an existing writable directory and are never overwritten without `-f`; stored names may only contain `A-Za-z0-9._ -` and no path separators; usernames are `A-Za-z0-9._-` (1-64 chars); passwords are 8-128 chars.

Web Routes (ICS0027)

- `GET /` - Main dashboard (file list; redirects to login when not authenticated)
- `GET/POST /auth/register` - Registration form and handler
- `GET/POST /auth/login` - Login interface and handler
- `POST /auth/logout` - End a session (CSRF-protected)
- `GET/POST /files/upload` - File upload form and backend processor (CSRF-protected)
- `GET /files/download/{id}` - Retrieve a file and stream its decrypted content
- `POST /files/delete/{id}` - Delete a file (CSRF-protected)

Sessions use an idle timeout of 15 minutes and an absolute expiry of 12 hours; the session cookie is `HttpOnly; Secure; SameSite=Strict` and is replaced at login. Sessions do not survive a server restart (the unlocked master key lives only in process memory).

Local Setup and Run Instructions

Prerequisites

- Python 3.11+
- `openssl` (always available on Linux/macOS) or `mkcert` (optional, preferred: certificate is trusted by your browser)

Environment Setup

```
# Clone the repository
git clone <repository-url>
cd <repository-folder>

# Create a virtual environment
python3 -m venv venv
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt
```

Running the CLI Tool

```
python -m src.cli.main --help
python -m src.cli.main register -u alice
python -m src.cli.main encrypt -u alice -i notes.txt -o notes.enc
python -m src.cli.main list -u alice
python -m src.cli.main decrypt -u alice -i notes.enc -o notes.out.txt
```

Running the Web Server (with TLS)

```
# Generate a local certificate (uses mkcert if installed, otherwise openssl)
./certs/generate_cert.sh

# Run the Uvicorn server over HTTPS
uvicorn src.web.main:app --host 127.0.0.1 --port 8000 --ssl-keyfile certs/key.pem --ssl-certfile certs/cert.pem
```

Open `https://localhost:8000` to access the application. With the `openssl` fallback the browser shows a self-signed certificate warning.

Data is stored under `./data` (SQLite database and encrypted blobs); set `SECFILE_DATA_DIR` / `SECFILE_STORAGE_DIR` to relocate it. Run all commands from the repository root.

Checkpoint 2 Progress Note (Week 9)

Implemented: CLI (register, login, encrypt, decrypt, list, wipe) with path/argument validation; AES-256-GCM file encryption with a per-user master key wrapped by an Argon2id-derived key; Argon2id password hashing; web interface (register, login, logout, dashboard, upload, download, delete) with server-side encryption/decryption, CSRF tokens, CSP and other security headers, server-side sessions with idle/absolute timeouts and session regeneration at login; HTTPS in the development setup via Uvicorn and `certs/generate_cert.sh`.

Still to do: automated tests, rate limiting / account lockout, streaming (chunked) encryption for large files, session-bound CSRF for the pre-login forms, and the final hardening review.